

LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 2
MODUL 12
“SEARCHING”



DISUSUN OLEH:
Dharma Chandra Viriya
109082500052
S1 IF-13-02
DOSEN:
Wahyu Andi Saputra, S.Pd., M.Eng.

PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2024/2025

DASAR TEORI

CONTOH SOAL

1. Latihan1

Source Code:

Output:

Deskripsi Program:

SOAL LATIHAN

Statement pencarian

1. Latihan1

Source Code:

```
package main

import "fmt"

func main() {
    var arr = [21]int{}
    totalVote := 0
    validVote := 0

    for {
        var suara int
        fmt.Scan(&suara)

        if suara == 0 {
            break
        }

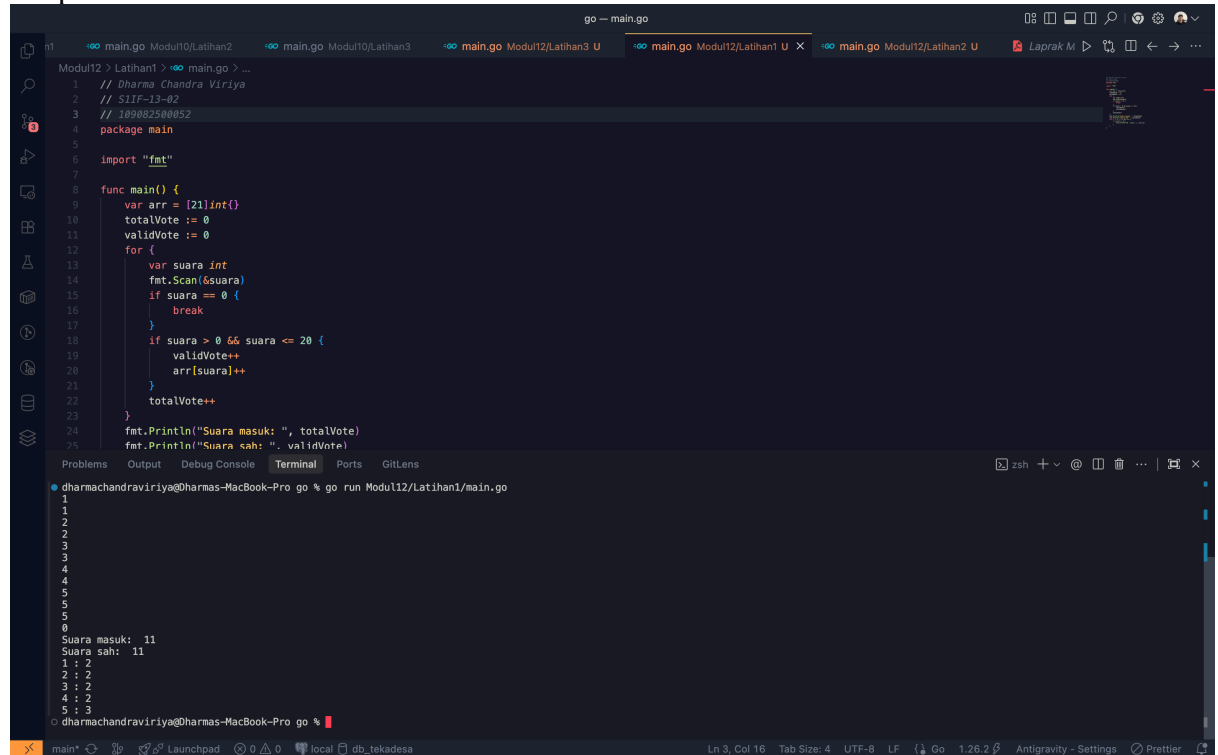
        if suara > 0 && suara <= 20 {
            validVote++
            arr[suara]++
        }

        totalVote++
    }

    fmt.Println("Suara masuk: ", totalVote)
    fmt.Println("Suara sah: ", validVote)

    for i := 1; i <= 20; i++ {
        if arr[i] > 0 {
            fmt.Printf("%d : %d\n", i, arr[i])
        }
    }
}
```

Output:



```
go -- main.go
Modul12 > Lathant1 > go main.go > ...
1 // Dharma Chandra Viriya
2 // SIIF-13-02
3 // 10500250052
4 package main
5
6 import "fmt"
7
8 func main() {
9     var arr = [21]int{}
10    totalVote := 0
11    validVote := 0
12    for {
13        var suara int
14        fmt.Scan(&suara)
15        if suara == 0 {
16            break
17        }
18        if suara > 0 && suara <= 20 {
19            validVote++
20            arr[suara]++
21        }
22        totalVote++
23    }
24    fmt.Println("Suara masuk: ", totalVote)
25    fmt.Println("Suara sah: ", validVote)
26}

Problems Output Debug Console Terminal Ports GitLens
dharmachandraviriya@Dharmas-MacBook-Pro go % go run Modul12/Lathant1/main.go
1
2
3
4
5
6
Suara masuk: 11
Suara sah: 11
1 : 2
2 : 2
3 : 2
4 : 2
5 : 3
dharmachandraviriya@Dharmas-MacBook-Pro go %
```

Deskripsi Program:

Program ini dibuat untuk membantu proses penghitungan suara pada pemilihan ketua RT yang memiliki 20 calon. Program menerima masukan berupa sejumlah nomor calon yang dipilih oleh warga. Data suara dibaca satu per satu hingga ditemukan angka `0` yang menandakan akhir masukan. Selain menghitung jumlah seluruh suara yang masuk, program juga melakukan validasi untuk memastikan bahwa nomor calon yang dipilih berada pada rentang 1 sampai 20. Suara yang berada di luar rentang tersebut dianggap tidak sah dan tidak dihitung sebagai suara valid. Setelah seluruh data selesai dibaca, program menampilkan jumlah suara yang masuk, jumlah suara sah, serta daftar calon yang memperoleh suara beserta jumlah suara yang diterimanya. Secara teknis, program menggunakan array `arr` berukuran 21 elemen untuk menyimpan frekuensi suara masing-masing calon, dengan indeks 1 sampai 20 digunakan sebagai nomor calon. Variabel `totalVote` digunakan untuk menghitung seluruh data suara yang masuk, baik valid maupun tidak valid, sedangkan `validVote` digunakan untuk menghitung jumlah suara yang sah. Proses pembacaan data dilakukan menggunakan perulangan tak hingga (`for`) yang akan berhenti ketika pengguna memasukkan nilai `0`. Setiap suara yang bernilai antara 1 hingga 20 akan menambah nilai `validVote` dan elemen array yang sesuai akan ditambah satu untuk mencatat suara calon tersebut. Setelah proses input selesai, program menampilkan jumlah suara masuk dan jumlah suara sah. Selanjutnya, program menelusuri array dari indeks 1 hingga 20 dan mencetak hanya calon yang memiliki jumlah suara lebih dari nol. Dengan memanfaatkan array sebagai pencacah frekuensi, proses penghitungan suara dapat dilakukan secara efisien dalam satu kali pembacaan data tanpa perlu melakukan pencarian atau pengurutan tambahan.

2. Latihan2

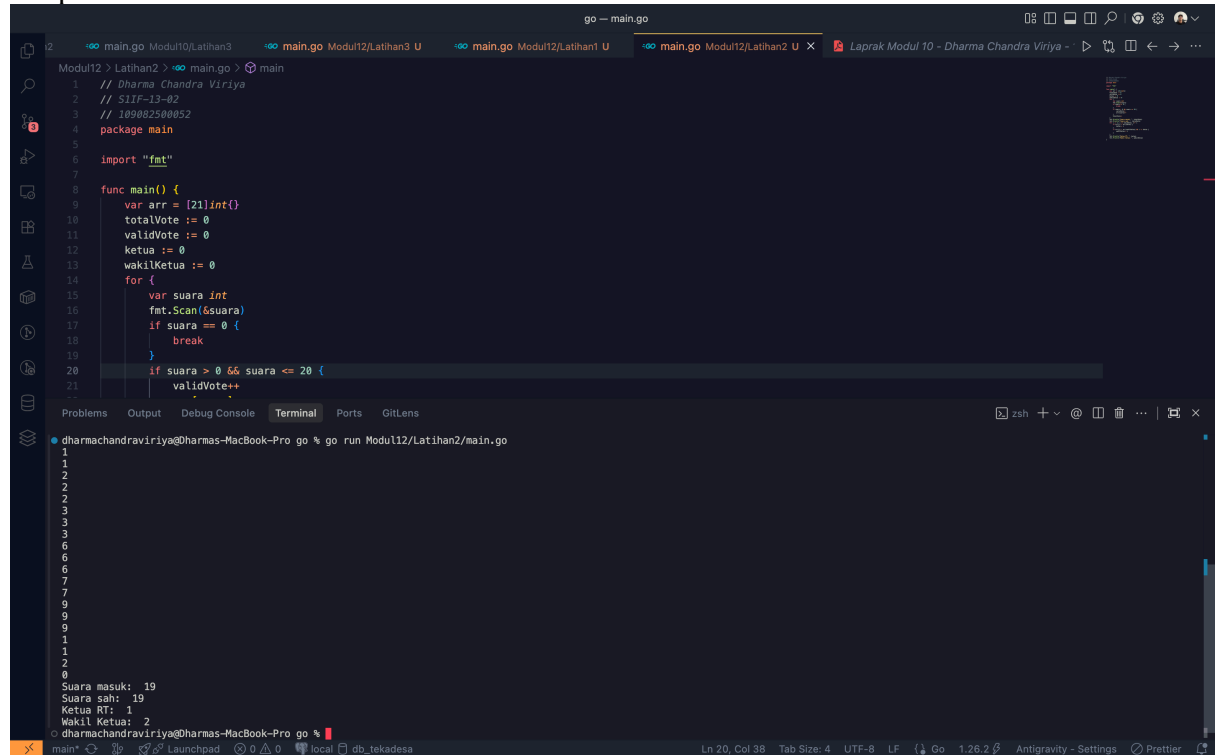
Source Code:

```
package main

import "fmt"

func main() {
    var arr = [21]int{}
    totalVote := 0
    validVote := 0
    ketua := 0
    wakilKetua := 0
    for {
        var suara int
        fmt.Scan(&suara)
        if suara == 0 {
            break
        }
        if suara > 0 && suara <= 20 {
            validVote++
            arr[suara]++
        }
        totalVote++
    }
    fmt.Println("Suara masuk: ", totalVote)
    fmt.Println("Suara sah: ", validVote)
    for i := 1; i <= validVote; i++ {
        if arr[i] > arr[ketua] {
            ketua = i
        }
        if arr[i] > arr[wakilKetua] && i != ketua {
            wakilKetua = i
        }
    }
    fmt.Println("Ketua RT: ", ketua)
    fmt.Println("Wakil Ketua: ", wakilKetua)
}
```

Output:



```
go -- main.go
Modul12 > Latihan2 > go main.go > main
1 // Dharma Chandra Viriya
2 // SIIP-13-02
3 // 10500250052
4 package main
5
6 import "fmt"
7
8 func main() {
9     var arr = [21]int{}
10    totalVote := 0
11    validVote := 0
12    ketua := 0
13    wakilKetua := 0
14    for {
15        var suara int
16        fmt.Scan(&suara)
17        if suara == 0 {
18            break
19        }
20        if suara > 0 && suara <= 20 {
21            validVote++
22            +
23        }
24    }
25    fmt.Println("Total suara masuk:", totalVote)
26    fmt.Println("Suara sah:", validVote)
27    fmt.Println("Ketua RT:", ketua)
28    fmt.Println("Wakil Ketua RT:", wakilKetua)
29 }
```

dharmachandraviriya@Dharmas-MacBook-Pro % go run Modul12/Latihan2/main.go

```
1
2
2
2
3
3
6
6
7
7
9
9
1
1
2
0
Suara masuk: 19
Suara sah: 19
Ketua RT: 1
Wakil Ketua RT: 2
dharmachandraviriya@Dharmas-MacBook-Pro %
```

Deskripsi Program:

Program ini digunakan untuk mengolah data hasil pemilihan ketua RT yang melibatkan 20 calon. Program menerima masukan berupa nomor calon yang dipilih oleh warga dan membaca data tersebut satu per satu hingga ditemukan angka 0 sebagai penanda akhir input. Setiap suara yang memiliki nilai antara 1 sampai 20 dianggap sebagai suara sah dan akan dihitung untuk calon yang bersangkutan, sedangkan nilai di luar rentang tersebut dianggap tidak valid. Setelah seluruh suara selesai diproses, program menampilkan jumlah total suara yang masuk, jumlah suara sah, serta menentukan calon yang memperoleh suara terbanyak sebagai ketua RT dan calon dengan perolehan suara terbanyak berikutnya sebagai wakil ketua RT. Untuk melakukan proses tersebut, program memanfaatkan array sebagai media penyimpanan jumlah suara setiap calon sehingga penghitungan suara dapat dilakukan secara efisien dan hasil pemilihan dapat diperoleh secara otomatis berdasarkan data yang dimasukkan.

3. Latihan3

Source Code:

```
package main

import "fmt"

const NMAX = 1000000

var data [NMAX]int

func isiArray(n int) {
    /* I.S. terdefinisi integer n, dan sejumlah n data
       sudah siap pada piranti masukan.
       F.S. array data berisi n bilangan yang dibaca
       dari piranti masukan. */

    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }
}

func posisi(n, k int) int {
    /* mengembalikan posisi k dalam array data dengan n elemen.
       Posisi dimulai dari 0. Jika tidak ada, kembalikan -1. */

    left := 0
    right := n - 1

    for left <= right {
        mid := (left + right) / 2

        if data[mid] == k {
            return mid
        } else if data[mid] < k {
            left = mid + 1
        } else {
            right = mid - 1
        }
    }

    return -1
}

func main() {
    /* I.S. terdefinisi bilangan bulat positif n dan k
       pada piranti masukan, serta n buah data integer
       positif yang sudah terurut membesar.
       F.S. tercetak posisi data k dalam array jika ada,
```

atau "TIDAK ADA" jika k tidak ditemukan. */

var n, k int

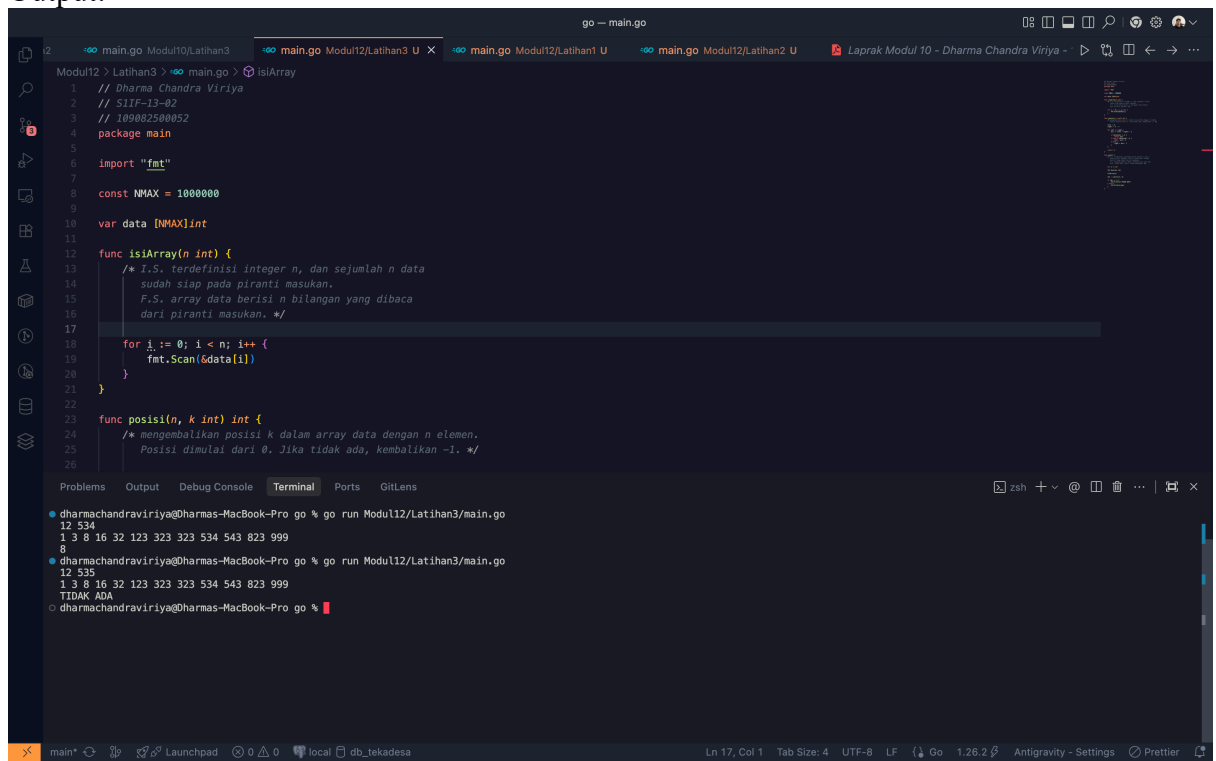
fmt.Scan(&n, &k)

isiArray(n)

idx := posisi(n, k)

```
if idx == -1 {  
    fmt.Println("TIDAK ADA")  
} else {  
    fmt.Println(idx)  
}  
}
```

Output:



The screenshot shows a Go IDE with a file named `main.go` in the `Modul12/Latihan3` directory. The code implements the `isiArray` and `posisi` functions. The `isiArray` function initializes an array of size `NMAX` (1,000,000) and fills it with random integers. The `posisi` function searches for a value `k` in the array and returns its index or -1 if not found. The terminal shows the output of running the program twice. In the first run, the input is `12 534` and the output is `1 3 8 16 32 123 323 323 534 543 823 999`. In the second run, the input is `12 535` and the output is `1 3 8 16 32 123 323 323 534 543 823 999` followed by `TIDAK ADA`.

```
1 // Dharma Chandra Viriya
2 // S1IF-13-02
3 // 100082500852
4 package main
5
6 import "fmt"
7
8 const NMAX = 1000000
9
10 var data [NMAX]int
11
12 func isiArray(n int) {
13     /* I.S. terdefinisi integer n, dan sejumlah n data
14     sudah siap pada piranti masukan.
15     F.S. array data berisi n bilangan yang dibaca
16     dari piranti masukan. */
17     for i := 0; i < n; i++ {
18         fmt.Scan(&data[i])
19     }
20 }
21
22 func posisi(n, k int) int {
23     /* mengembalikan posisi k dalam array data dengan n elemen.
24     Posisi dimulai dari 0. Jika tidak ada, kembalikan -1. */
25     // ... (search logic) ...
26 }
```

Terminal Output:

```
dharmachandraviriya@Dharmas-MacBook-Pro go % go run Modul12/Latihan3/main.go
12 534
1 3 8 16 32 123 323 323 534 543 823 999
8

dharmachandraviriya@Dharmas-MacBook-Pro go % go run Modul12/Latihan3/main.go
12 535
1 3 8 16 32 123 323 323 534 543 823 999
TIDAK ADA
```

Deskripsi Program:

Program ini dibuat untuk mencari posisi suatu bilangan dalam kumpulan data yang telah terurut secara menaik. Pengguna memasukkan banyaknya data (n), nilai yang ingin dicari (k), serta sejumlah n bilangan bulat yang sudah berada dalam keadaan terurut. Program kemudian melakukan pencarian terhadap nilai k di dalam array dan menampilkan posisi indeksinya apabila ditemukan. Jika nilai tersebut tidak terdapat dalam array, program akan menampilkan tulisan **"TIDAK ADA"** sebagai keluaran. Secara teknis, program menggunakan array global `data` dengan kapasitas maksimum satu juta elemen untuk menyimpan seluruh data masukan. Proses pengisian array dilakukan oleh prosedur `isiArray`, yang membaca sebanyak n bilangan dari piranti masukan dan menyimpannya ke dalam array. Pencarian data dilakukan oleh fungsi `posisi` menggunakan algoritma **Binary Search**. Algoritma ini bekerja dengan membandingkan nilai yang dicari dengan elemen tengah array, kemudian mempersempit ruang pencarian ke bagian kiri atau kanan array sesuai hasil perbandingan. Proses tersebut dilakukan berulang hingga data ditemukan atau ruang pencarian habis. Jika data ditemukan, fungsi mengembalikan indeks posisi data dalam array yang dimulai dari 0. Sebaliknya, jika data tidak ditemukan, fungsi mengembalikan nilai `-1`. Pada fungsi `main`, hasil pencarian diperiksa dan ditampilkan kepada pengguna, baik berupa posisi data maupun pesan bahwa data tidak ditemukan. Penggunaan algoritma Binary Search membuat proses pencarian menjadi lebih efisien dibandingkan pencarian linear karena hanya memerlukan kompleksitas waktu **$O(\log n)$** pada data yang sudah terurut.

DAFTAR PUSTAKA